Klasy i funkcje – Gra Memory (Android/Java)

Poniżej opisano wszystkie klasy i metody potrzebne do realizacji projektu. Przy każdej omówiono **cel istnienia**, **działanie** i **powiązania z innymi elementami**.

---

1. Klasa `Card` – model danych jednej karty

Cel: Przechowuje stan pojedynczej karty w grze. Nie wie nic o interfejsie graficznym – to czysty model danych.

Pola klasy:

Pole	Typ	Opis
`value`	`int`	Numer karty (1–4), odpowiada nazwie pliku graficznego (`card_1.png` itd.)
`isRevealed`	`boolean`	Czy karta jest aktualnie odkryta (pokazuje awers). Domyślnie `false`.
`isMatched`	`boolean`	Czy karta została już trwale dopasowana. Domyślnie `false`.

Metody klasy:

`Card(int value)`

Konstruktor. Przyjmuje wartość karty i ustawia obie flagi na `false`. Dzięki temu każda nowo utworzona karta zaczyna jako zakryta i niepasowana.

`getValue()`

Getter zwracający wartość `value`. Używany przy porównywaniu dwóch odkrytych kart – logika gry pyta każdą kartę o jej wartość i porównuje wyniki.

`isRevealed()` / `setRevealed(boolean)`

Getter i setter dla flagi `isRevealed`. `setRevealed(true)` wywołujemy gdy gracz kliknie kartę. `setRevealed(false)` wywołujemy gdy dwie karty nie pasują i wracają do rewersu.

`isMatched()` / `setMatched(boolean)`

Getter i setter dla flagi `isMatched`. Po poprawnym dopasowaniu wywołujemy `setMatched(true)` – od tej pory karta jest „zamrożona" i nie reaguje na kliknięcia.

> Gdybyśmy trzymali stan kart bezpośrednio w `MainActivity` (np. jako tablice intów i booleanów), kod szybko stałby się nieczytelny. Klasa `Card` grupuje powiązane dane w jednym miejscu – to zasada **Single Responsibility**

---

2. Klasa `MainActivity` – centrum dowodzenia

Cel: Główna (i jedyna) aktywność aplikacji. Odpowiada za wyświetlanie interfejsu, obsługę kliknięć i koordynację logiki gry.

2a. Pola klasy (atrybuty instancji)

Przed opisem metod ważne jest zrozumienie, jakie dane `MainActivity` przechowuje przez cały czas życia:

Pole	Typ	Cel
`cards`	`Card[]`	Tablica 8 obiektów `Card` – pełen stan logiczny planszy
`imageViews`	`ImageView[]`	Tablica 8 referencji do widoków kart na ekranie
`firstIndex`	`int`	Indeks pierwszej odkrytej karty w bieżącym ruchu (`-1` = brak)
`secondIndex`	`int`	Indeks drugiej odkrytej karty w bieżącym ruchu (`-1` = brak)
`moves`	`int`	Licznik ruchów gracza (inkrementowany po odkryciu drugiej karty)
`isChecking`	`boolean`	Blokada kliknięć podczas sprawdzania dopasowania i odwracania kart
`tvMoves`	`TextView`	Referencja do pola tekstowego wyświetlającego liczbę ruchów

> Pola klasy (nie zmienne lokalne metod!) są dostępne z każdej metody. `firstIndex` musi przetrwać pomiędzy dwoma oddzielnymi kliknięciami – dlatego nie może być zmienną lokalną.

2b. Metody cyklu życia

`onCreate(Bundle savedInstanceState)`

Pierwsza metoda wywoływana przez system Android przy starcie aplikacji. Tu inicjalizujemy wszystko co potrzebne:

-> Wywołujemy `setContentView()` wskazując nasz layout XML

-> Wywołujemy `initViews()` – podpinamy widoki

-> Wywołujemy `startNewGame()` – startujemy pierwszą grę

Nie umieszczamy tu logiki gry – delegujemy ją do wyspecjalizowanych metod.

2c. Metody inicjalizacji

`initViews()`

Jej jedynym zadaniem jest znalezienie elementów UI w drzewie widoków i zapamiętanie ich w polach klasy.

Działanie krok po kroku:

1. Iteruje od 0 do 7 i wywołuje `findViewById()` dla każdego `ImageView`, wypełniając tablicę `imageViews[]`
2. Pobiera referencję do `TextView` z licznikiem ruchów
3. Pobiera referencję do `Button` i ustawia mu `OnClickListener` wywołujący `startNewGame()`

> Używamy `R.id.iv_card_0`, `R.id.iv_card_1` itd. – identyfikatory zdefiniowane w XML. Można to uprościć budując identyfikator dynamicznie przez `getResources().getIdentifier()`, ale dla 8 kart prosta pętla z tablicą ID jest czytelniejsza.

`startNewGame()`

Resetuje całą grę do stanu początkowego. Wywoływana na starcie aplikacji i po każdym zakończeniu gry.

Działanie krok po kroku:

1. **Tworzy listę wartości** `ArrayList<Integer>` zawierającą `[1,1,2,2,3,3,4,4]`
2. **Tasuje listę** – używa `Collections.shuffle()`, który implementuje algorytm losowego przestawienia. Przyjmuje obiekt `List` i opcjonalnie `Random`. Modyfikuje listę **w miejscu** (nie zwraca nowej).
3. **Wypełnia tablicę** `cards[]` – iteruje po liście i tworzy nowy obiekt `Card` dla każdej pozycji
4. **Resetuje zmienne stanu** – `firstIndex = -1`, `secondIndex = -1`, `isChecking = false`, `moves = 0`
5. **Aktualizuje UI** – odświeża `TextView` z liczbą ruchów
6. **Renderuje planszę** – wywołuje `renderAllCards()`

2d. Metody renderowania (wyświetlania)

`renderAllCards()`

Prosta metoda iterująca po wszystkich 8 pozycjach i wywołująca `loadCardImage()` dla każdej pary `(imageView, card)`.

Używana przy inicjalizacji i po powrocie kart do rewersu (wtedy chcemy odświeżyć całą planszę naraz).

`loadCardImage(ImageView imageView, Card card)`

Kluczowa metoda odpowiedzialna za wyświetlenie właściwej grafiki na danym widoku.

Działanie:

1. **Decyduje o nazwie pliku** na podstawie stanu karty:
   - `card.isMatched()` → karta może być niewidoczna lub pokazywać awers na stałe (zależnie od decyzji projektowej)
   - `card.isRevealed()` → buduje String `"card_" + card.getValue() + ".png"` (np. `"card_3.png"`)
   - inaczej → używa Stringa `"card_back.png"`
2. **Wczytuje plik z** `AssetManager` – pobierany przez `getAssets()`. Metoda `open(String fileName)` zwraca `InputStream`.
3. **Dekoduje strumień do bitmapy** – `BitmapFactory.decodeStream(InputStream)` konwertuje surowe bajty obrazu na obiekt `Bitmap` gotowy do wyświetlenia.
4. **Ustawia bitmapę** na `ImageView` przez `setImageBitmap(Bitmap)`.
5. **Zamyka strumień** – `InputStream.close()` w bloku `finally` lub przez `try-with-resources`. Niezamknięty strumień to wyciek zasobów.

> Metoda `getAssets().open()` rzuca `IOException` – obowiązkowo obsłuż przez `try-catch`. W catch możesz zalogować błąd przez `Log.e("MemoryGame", "Błąd ładowania: " + fileName, e)`.

2e. Metody logiki gry

`onCardClicked(int index)`

Wywoływana przez `OnClickListener` każdego `ImageView`. Parametr `index` to pozycja karty w tablicy (0–7).

Działanie – seria warunków obronnych (tzw. *guard clauses*):

Jeśli `isChecking == true` → **wyjdź** (trwa animacja powrotu kart)

Jeśli `cards[index].isMatched()` → **wyjdź** (karta już dopasowana)

Jeśli `cards[index].isRevealed()` → **wyjdź** (ta sama karta kliknięta ponownie)

Odkrywa kartę: `setRevealed(true)` + `loadCardImage()`

Następnie sprawdza który to wybór w ruchu:

- Jeśli `firstIndex == -1` → zapamiętuje jako pierwszy, czeka na drugi wybór
- Jeśli `firstIndex != -1` → to drugi wybór: zapamiętuje, zwiększa `moves`, aktualizuje `TextView`, wywołuje `checkMatch()`

`checkMatch()`

Sprawdza czy dwie odkryte karty pasują do siebie i odpowiednio reaguje.

Przypadek 1: Karty pasują (`cards[firstIndex].getValue() == cards[secondIndex].getValue()`)

- Ustawia `isMatched(true)` dla obu kart
- Resetuje `firstIndex` i `secondIndex` do `-1`
- Ustawia `isChecking = false`
- Wywołuje `checkWinCondition()`

Przypadek 2: Karty nie pasują

- Ustawia `isChecking = true` (blokuje kliknięcia)
- Tworzy obiekt `Handler` powiązany z `Looper.getMainLooper()` (wątek UI)
- Wywołuje `postDelayed(Runnable, long)` z opóźnieniem 1000ms

Wewnątrz `Runnable` (kod wykonany po sekundzie):

- Ustawia `isRevealed(false)` dla obu kart
- Wywołuje `loadCardImage()` dla obu kart (powrót do rewersu)
- Resetuje indeksy
- Ustawia `isChecking = false`

> `Handler` Android nie pozwala blokować głównego wątku (UI thread). `Thread.sleep(1000)` zamroziłoby cały interfejs – ekran przestałby reagować. `Handler.postDelayed()` planuje wykonanie kodu **w przyszłości** bez blokowania, a interfejs działa normalnie przez ten czas.

`checkWinCondition()`

Sprawdza czy gra została ukończona.

Działanie:

Iteruje przez tablicę `cards[]` i sprawdza czy każda karta ma `isMatched() == true`

Jeśli znajdzie choć jedną kartę z `isMatched() == false` → wychodzi (gra trwa)

Jeśli wszystkie dopasowane → tworzy i wyświetla `AlertDialog`

`AlertDialog.Builder` – budowniczy okna dialogowego:

- `.setTitle()` – tytuł okna
- `.setMessage()` – treść (tu wstawiamy liczbę ruchów przez `getString(R.string.game_won_message, moves)`)
- `.setPositiveButton()` – przycisk z `OnClickListener` wywołującym `startNewGame()`
- `.setCancelable(false)` – gracz nie może zamknąć dialogu przyciskiem „wstecz"
- `.show()` – wyświetla okno

---

3. Klasy i metody systemowe – wyjaśnienie

`Collections.shuffle(List<?> list, Random rnd)`

Należy do `java.util.Collections`. Implementuje tasowanie Fisher-Yates: iteruje od końca listy do początku i każdy element zamienia miejscami z losowym elementem przed nim. Gwarantuje równomierny rozkład wszystkich permutacji.

`AssetManager`

Klasa systemowa Androida do dostępu do plików z folderu `assets/`. Kluczowe metody:

- `open(String fileName)` – otwiera plik, zwraca `InputStream`. Jeśli plik nie istnieje, rzuca `IOException`.
- `list(String path)` – zwraca tablicę nazw plików w podanym podkatalogu (przydatne przy rozbudowie gry o więcej poziomów).

`BitmapFactory`

Klasa narzędziowa do dekodowania grafik. Metoda `decodeStream(InputStream is)` czyta strumień bajtów i tworzy obiekt `Bitmap`. Nie rzuca wyjątku – zwraca `null` gdy dekodowanie się nie powiedzie, więc warto sprawdzić wynik przed użyciem.

`Handler` + `Looper`

`Handler` to mechanizm kolejkowania zadań w wątku. `Looper.getMainLooper()` wskazuje na pętlę zdarzeń głównego wątku UI. Metoda `postDelayed(Runnable r, long delayMillis)` dodaje zadanie `r` do kolejki z opóźnieniem. Po upływie czasu Android sam wywoła `r.run()` w odpowiednim wątku.

`Log`

Klasa do logowania w Android Studio (widoczne w oknie Logcat):

- `Log.d(tag, message)` – debug
- `Log.e(tag, message, throwable)` – błąd z wyjątkiem

Zamiast `System.out.println()` – zawsze używaj `Log` W Androidzie.

---

4. Schemat zależności między klasami

```
MainActivity
│
├── posiada tablicę ──► Card[] (8 obiektów klasy Card)
│                   └── value, isRevealed, isMatched
│
├── posiada tablicę ──► ImageView[] (8 widoków z layoutu)
│
├── używa ──► AssetManager (wczytuje pliki z assets/)
│         └── zwraca InputStream
│               └── BitmapFactory.decodeStream()
│                     └── zwraca Bitmap → ImageView.setImageBitmap()
│
├── używa ──► Collections.shuffle() (tasowanie listy wartości)
│
├── używa ──► Handler + Looper (opóźnione odwrócenie kart)
│
└── wyświetla ──► AlertDialog.Builder (okno wygranej)
```

5. Przepływ danych – jeden pełny ruch gracza

```
Kliknięcie ImageView[3]
      |
      ▼
onCardClicked(3)
   ├─ sprawdź isChecking, isMatched, isRevealed  ← guard clauses
   ├─ cards[3].setRevealed(true)
   ├─ loadCardImage(imageViews[3], cards[3])     ← wczyta "card_2.png"
  └─ firstIndex = 3

Kliknięcie ImageView[6]
      |
      ▼
onCardClicked(6)
   ├─ cards[6].setRevealed(true)
   ├─ loadCardImage(imageViews[6], cards[6])     ← wczyta "card_4.png"
   ├─ moves++  →  tvMoves.setText(...)
  └─ checkMatch()
        |
                ├─ cards[3].value(2) ≠ cards[6].value(4)
                ├─ isChecking = true
        └─ Handler.postDelayed(1000ms)
              |
              ▼ (po 1 sekundzie)
          cards[3].setRevealed(false)
          cards[6].setRevealed(false)
          loadCardImage() dla obu      ← wróci "card_back.png"
          firstIndex = -1
          isChecking = false
```